

Procedures Used in Coupling UEDGE & DEGAS 2 And Possible Generalizations

The existing iterative procedure for running UEDGE coupled to DEGAS 2 proceeds as follows:

1. Run UEDGE
 - Initially with its built-in neutral transport model.
2. Call the UEDGE subroutine `writemcnfile` to write the text file to be passed to DEGAS 2 → `uedata`,
 - Roughly speaking, this file can be broken down into sections:
 - Indexing information: mesh size, number & location of X-points.
 - Grid: R , Z values of all 4 corners & center of each cell.
 - Magnetic field vector at each of these points (not used in DEGAS 2).
 - Ion density at all mesh centers, looped over ion species.
 - Cylindrical R velocity (equivalent to Cartesian x at $\phi = 0$) at all mesh centers, looped over ion species.
 - Cylindrical Z velocity at all mesh centers, looped over ion species.
 - Cylindrical ϕ velocity (equivalent to Cartesian y at $\phi = 0$) at all mesh centers, looped over ion species.
 - Ion temperature (common to all ion species) at all mesh centers.
 - Electron temperature at all mesh centers.
 - For each X-point:
 - Ion density along inner target, looped over ion species.
 - * This & subsequent quantities are used to set sheath boundary conditions.
 - R velocity along inner target, looped over ion species.
 - Z velocity along inner target, looped over ion species.
 - ϕ velocity along inner target, looped over ion species.
 - Common ion temperature along inner target.
 - Electron temperature along inner target.
 - Ion current to inner target, looped over ion species.
 - Same list of quantities for outer target.
3. Prior to initial run, set up reference & problem data for DEGAS 2.
4. Set up `definegeometry2d` input file, using `uedge_mesh` keyword, to read in `uedata` file.
 - The subroutine `read_uedge_mesh` in `definegeometry2d` is used to read the `uedata` file,
 - Only the indexing & grid information is read,
 - The grid is written out in Sonnet format to a file `uedge_sonnet` for the user's convenience, e.g., for loading into DG.
 - The mesh indices are retained within DEGAS 2 and are used in writing the output so that the two can be related.

- The simplest possible input file would define additional solid polygons around the UEDGE mesh and use its boundaries as the “walls”.
 - Alternatively, one can use the actual machine hardware to define the solid polygons & set up intervening vacuum regions in between.
 - Either way, one can likely arrange it so that these input files are general & need not be revised with every new run.
 - Moreover, in an iterative calculation, `definegeometry2d` need only be run prior to the initial execution of DEGAS 2.
5. Run `definegeometry2d`.
6. Set up input file for `defineback` using the `plasma_file` keyword,
- For iterative calculations:
 - The `iterative_run` keyword should be given.
 - The file name passed to `plasma_file` should end in “.u”,
 - * This lets `defineback` know that you wish (& need) to set all plasma & source data using the `uedata` file.
 - * The file is read & processed by subroutine `read_uedge_background` & callees in `writeback.web`.
 - A valid path & file name should be set for `oldsourcefile` in `degas2.in`,
 - * This causes the code to write out the current source specification to this file near the end of `defineback` execution,
 - * On subsequent iterations, the file is read back in & used to enforce changes in the source currents via weighting factors (see the “Iterative Problems” section of the `defineback` documentation).
 - When using DEGAS 2 as a post-processor for UEDGE:
 - One can follow the `plasma_file` keyword with additional sources (e.g., a gas puff).
 - If the file pointed to by `plasma_file` does not end in “.u”, it is assumed to have the format described in the documentation for `readbackground`, although the processing is actually done in subroutine `init_uedge_background` in `writeback.web`.
 - This file must use the `uedge_file` keyword to point to the `uedata` file.
 - The user can also include other keywords to specify plasma parameters in the “vacuum” regions between the UEDGE mesh and the walls.
 - Note that these capabilities would give rise to unbalanced sources & sinks if the DEGAS 2 output were passed back to UEDGE.
7. Run `defineback` with a suitable number of flights.
- The number of flights could be altered from the default 100 in an automated fashion via a script or by altering the code in the `read_uedge_background` subroutine.
 - For iterative calculations, the same input file can be used (assuming the name used for the “uedata” file is the same).
8. Run `tallysetup` with a standard input file,
- The tallies used in assembling the data passed to UEDGE are as close to “defaults” as tallies get in DEGAS 2; they appear in all practical input files.

9. Run `flighttest`,

- Near the end of the run, this calls subroutine `fill_coupling_arrays` in `doflights.web`,
- This writes out the tallies for test species density, temperature, and flux to the `testdata.out` file,
 - There is an outer loop over test species,
 - And inner loops over the grid indices.
 - When UEDGE is run next, this file is read by subroutine `readmcntest`.
- And the tallies “ion source rate”, “ion momentum source vector”, and “ion energy source”,
 - The last is summed over all ion species.
 - There is an outer loop over source group, i.e., one source group for each divertor target.
- When UEDGE is run next, this file is read by subroutine `readmcnsor`.

10. When iterating, steps 1, 2, 7, and 9 are performed repeatedly until the UEDGE residuals reach the desired level.

Of the capabilities built into DEGAS 2, the only one that is not readily generalized for use with other codes or geometries is the `read_uedge_background` routine. Reading the plasma data and mapping it onto the primary grid is relatively general, but defining the recycling sources at the target plates and setting the sheath parameters there are not. Certainly, one could use `read_uedge_background` as a template for writing a new routine, but it might also be possible to use a default or simple `get_n_t` routine to handle the plasma data and then utilize `defineback`’s existing formats to set the recycling source, etc.

Although the UEDGE mesh is naturally two-dimensional, the tools used in DEGAS 2 for handling it can be (and have been already for XGC0) generalized for use with an unstructured mesh by making the second dimension trivial. This is because DEGAS 2 makes no inferences about mesh connectivity from the indices; they are just labels.